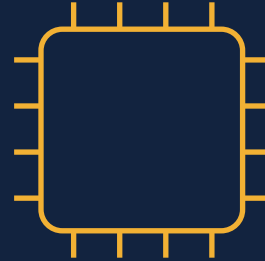


SILICON BADGERS

SystemVerilog Onboarding Guide



HDLBits practice, SystemVerilog basics, and three starter projects

WHAT YOU WILL DO

1. Complete 25 selected HDLBits exercises
2. Learn the small Verilog-to-SystemVerilog differences
3. Build three RTL projects

Audience: Students who have completed ECE 252

Scope: Synthesizable RTL only

START HERE

Three steps, then you can build

You already know digital logic from ECE 252. You do not need to learn all of SystemVerilog. Work through these three steps and focus on small synthesizable circuits.

1**Practice the circuit basics in HDLBits**

Complete the selected Verilog exercises on signals, modules, combinational logic, registers, counters, and one FSM.

2**Switch to the club's SystemVerilog style**

Use logic, always_comb, always_ff, named port connections, and readable FSM state names.

3**Build three small projects**

Build a signed ReLU, a registered accumulator, and a four-element dot product.

The only question to keep asking

Does this output depend only on the inputs right now, or does it need to remember something from an earlier clock cycle?

CORE IDEA

Combinational vs. sequential logic

Almost every beginner RTL block fits into one of these two categories. Decide which one you need before writing code.

Combinational logic

Think of it like a calculator: change the inputs and the answer changes.

- It does not remember an old value.
- It does not need a clock.
- Examples: gates, muxes, adders, comparisons, and ReLU.
- Use assign or always_comb.
- Assign every output in every possible path.

```
always_comb begin
  if (a > b)
    y = a;
  else
    y = b;
end
```

Sequential logic

Think of it like memory: the circuit stores a value and updates it on a clock edge.

- It remembers a value from an earlier cycle.
- It needs a clock.
- Examples: registers, counters, accumulators, and FSM state.
- Use always_ff @(posedge clk).
- Use nonblocking assignments (<=).

```
always_ff @(posedge clk) begin
  if (reset)
    count <= 0;
  else if (enable)
    count <= count + 1;
end
```

Simple rule

Use = in always_comb. Use <= in always_ff. Use a clock only when the circuit must remember something.

HDLBITS

Complete these 25 exercises

Do them in order. HDLBits uses Verilog; the next part of this guide shows how to write the same ideas in SystemVerilog.

A. Signals and vectors

1. Getting Started	2. Simple wire
3. Inverter	4. AND gate
5. Declaring wires	6. Vectors
7. Vector part select	8. Bitwise operators
9. Vector concatenation operator	

B. Modules and combinational logic

10. Modules	11. Connecting ports by name
12. Modules and vectors	13. Adder 1
14. Adder-subtractor	15. Always blocks (combinational)
16. If statement	17. Case statement
18. Avoiding latches	19. 2-to-1 bus multiplexer
20. Signed addition overflow	

C. Registers and control

21. Always blocks (clocked)	22. DFF with reset
23. Four-bit binary counter	24. 4-bit shift register
25. Simple FSM 1 (synchronous reset)	

Exercise index: hdlbits.01xz.net/wiki/Problem_sets

PART 3

Verilog to SystemVerilog

SystemVerilog extends Verilog. Most HDLBits solutions remain conceptually correct, but the club will use clearer constructs that state whether logic is combinational or clocked.

1. Use logic for signals

HDLBits Verilog	Club SystemVerilog
<pre>module example (input wire a, output reg y);</pre>	<pre>module example (input logic a, output logic y);</pre>
Use logic for ports and internal signals in beginner club RTL. A signal must still have only one driver.	

2. Use always_comb for combinational procedures

HDLBits Verilog	Club SystemVerilog
<pre>always @(*) begin if (select) y = a; else y = b; end</pre>	<pre>always_comb begin if (select) y = a; else y = b; end</pre>
always_comb automatically includes every signal read by the block and communicates combinational intent.	

3. Use always_ff for registers

HDLBits Verilog	Club SystemVerilog
<pre>always @(posedge clk) begin if (reset) q <= 0; else q <= d; end</pre>	<pre>always_ff @(posedge clk) begin if (reset) q <= 0; else q <= d; end</pre>
The club uses rising-edge registers, active-high synchronous reset, and nonblocking assignments.	

SYSTEMVERILOG BRIDGE

Arrays, signed values, and FSMs

Packed and unpacked arrays

```
logic [7:0] byte_value;           // One 8-bit packed value
logic [7:0] samples [0:3];       // Four unpacked 8-bit values
logic signed [7:0] signed_sample; // One signed 8-bit value
```

Bits written before the name form one packed number. A range after the name creates multiple elements. The dot-product project uses an unpacked array of signed packed values.

Signed arithmetic

Declare values signed when negative numbers are possible. Multiplication and accumulation require wider results. Two signed 8-bit values can produce a signed 16-bit product, and several products need additional accumulator bits. Never assume the tool will choose the width you intended.

```
logic signed [7:0] a;
logic signed [7:0] b;
logic signed [15:0] product;
logic signed [19:0] sum;

always_comb begin
    product = a * b;
end
```

Finite-state machines

SystemVerilog can give states readable names. Use one clocked block for the current state and one combinational block for the next state and outputs.

```
typedef enum logic [1:0] {
    IDLE,
    RUN,
    DONE
} state_t;

state_t state;
state_t next_state;

always_ff @(posedge clk) begin
    if (reset)
        state <= IDLE;
    else
        state <= next_state;
end

always_comb begin
    next_state = state;

    case (state)
        IDLE: if (start) next_state = RUN;
        RUN:  if (last)  next_state = DONE;
        DONE:      next_state = IDLE;
        default:    next_state = IDLE;
    endcase
end
```

SYSTEMVERILOG BRIDGE

Club RTL conventions

Rule	Club convention
Signals	Use logic for ports and internal signals.
Combinational	Use always_comb and blocking assignments (=). Assign every output on every path.
Clocked	Use always_ff @(posedge clk) and nonblocking assignments (<=).
Reset	Use an active-high synchronous reset unless a module specification says otherwise.
Connections	Connect module ports by name, such as .clk(clk), rather than by position.
State	Use typedef enum logic for readable FSM state names.
Widths	Write explicit widths and signed declarations. Treat truncation warnings as real problems.
Formatting	Use descriptive names, one assignment per line, and simple control flow.

Common mistakes

- Incomplete combinational assignment: An output keeps its previous value and creates an unintended latch.
- Blocking assignment in clocked logic: Simulation ordering may not represent the intended registers.
- Nonblocking assignment in combinational logic: The block becomes harder to reason about and may simulate unexpectedly.
- Unsigned declaration: A negative two's-complement value is treated as a large positive value.
- Narrow accumulator: Valid high bits are silently discarded.
- Multiple drivers: Two blocks attempt to control the same signal.

Not covered yet: classes, interfaces, packages, assertions, tasks, functions, parameters, generate blocks, clock-domain crossing, or UVM. These are useful later, but they are not required for the starter projects.

PART 4

Project 1: Signed ReLU

Goal

Build a small combinational block that behaves like the ReLU activation used by the mini-TPU.

Required behavior

- Create a module named `relu8`.
- Use one signed 8-bit input named `in_value`.
- Use one signed 8-bit output named `out_value`.
- If the input is negative, output zero. Otherwise, copy the input.
- Use `logic` and `always_comb`.
- Do not use a clock, state, tasks, functions, or parameters.

Pass condition: The provided checker passes -128, -1, 0, 1, and 127, with no inferred latch warnings.

Starter interface

```
module relu8 (  
    input  logic signed [7:0] in_value,  
    output logic signed [7:0] out_value  
);  
  
    // Your RTL here  
  
endmodule
```

Before submitting

- Explain why `8'b11111111` represents -1 when declared signed.
- Show that every branch assigns `out_value`.
- Draw the comparator and multiplexer represented by your RTL.

PART 4

Project 2: Registered accumulator

Goal

Build a clocked block that adds one signed input into a wider running sum whenever enable is high.

Required behavior

- Create a module named accumulator8.
- Use clk, reset, and enable inputs.
- Use one signed 8-bit input named in_value.
- Use one signed 19-bit output named sum.
- On a rising clock edge, reset sets sum to zero.
- Otherwise, enable adds in_value to sum; when enable is low, sum holds its value.
- Use always_ff and nonblocking assignments.

Pass condition: The provided checker verifies reset, positive values, negative values, disabled cycles, and repeated additions.

Starter interface

```
module accumulator8 (  
    input  logic          clk,  
    input  logic          reset,  
    input  logic          enable,  
    input  logic signed [7:0] in_value,  
    output logic signed [18:0] sum  
);  
  
    // Your RTL here  
  
endmodule
```

Before submitting

- Explain why sum is wider than in_value.
- Show the expected sum after adding 5, -2, and 7 on three enabled cycles.
- Explain what happens when enable is low.

PART 4

Project 3: Four-element dot product

Goal

Combine signed multiplication, accumulation, a counter, and control signals into a small multi-cycle datapath.

Required behavior

- Create a module named `dot_product4`.
- Accept two arrays containing four signed 8-bit values each.
- When start is accepted, clear the accumulator and process one pair per cycle.
- Use one multiplier and one accumulator rather than four parallel multipliers.
- Raise done for one clock cycle after all four products have been accumulated.
- Ignore a new start while the module is busy.
- Use a signed 19-bit or wider result.

Pass condition: The provided checker verifies positive, negative, zero, extreme, reset, and back-to-back calculations.

Starter interface

```
module dot_product4 (
    input  logic          clk,
    input  logic          reset,
    input  logic          start,
    input  logic signed [7:0] a [0:3],
    input  logic signed [7:0] b [0:3],
    output logic          busy,
    output logic          done,
    output logic signed [18:0] result
);

    // Your RTL here

endmodule
```

Suggested internal pieces

- A 2-bit index selecting array elements 0 through 3.
- A signed product signal wide enough for 8-bit by 8-bit multiplication.
- A signed accumulator wide enough for four products.
- IDLE and RUN states, plus output logic for busy and done.

CHECK YOUR UNDERSTANDING

Review questions

1. Why can hardware description statements operate concurrently even though they are written on separate lines?
2. Rewrite output reg [7:0] y using the club's SystemVerilog style.
3. Which assignment operator should be used in always_comb? Which should be used in always_ff?
4. What hardware may be inferred if a combinational block fails to assign an output on one path?
5. What is the difference between logic [7:0] samples [0:3] and logic [3:0][7:0] samples?
6. What decimal value does logic signed [7:0] x = 8'b11111111 represent?
7. Two signed 8-bit numbers are multiplied. What is the minimum safe width for the full product?
8. Why does the dot-product module need state or a counter if it uses only one multiplier?

Do not look at the answer key until you can explain each answer aloud. The goal is not memorizing syntax; it is connecting syntax to the hardware being created.

REVIEW QUESTIONS

Answer key

1. The code describes separate pieces of hardware that exist and react at the same time; source-code order does not create a software-style execution sequence.
2. Use output logic `[7:0] y`. The logic type replaces the beginner-level wire/reg distinction in club RTL, while the driving construct still determines behavior.
3. Use blocking assignment (`=`) in `always_comb` and nonblocking assignment (`<=`) in `always_ff`.
4. The tool may infer a latch because the output must remember its previous value on the missing path.
5. `logic [7:0] samples [0:3]` is an unpacked array of four 8-bit elements. `logic [3:0][7:0] samples` is one 32-bit packed value divided into packed dimensions.
6. It represents -1 in signed two's-complement form.
7. A full signed 8-bit by 8-bit product requires 16 bits.
8. One multiplier handles only one pair per cycle, so the circuit must remember which pair is active and when all four pairs are finished.

Ready-to-contribute checklist

- ☐ All 25 assigned HDLBits problems show Success.
- ☐ You can explain `always_comb`, `always_ff`, `=`, and `<=` without looking them up.
- ☐ You completed the three projects using synthesizable SystemVerilog.
- ☐ You can explain the width and signedness of every arithmetic signal you created.
- ☐ You can read a mismatch waveform and identify the first incorrect cycle.
- ☐ Another club member reviewed your dot-product RTL.

AFTER ONBOARDING

What comes next

Members who complete this guide are ready for small, supervised tasks in the V1 mini-TPU. They are not expected to understand the entire accelerator immediately. Begin with modules that have a short specification and a provided checker.

Starting area	Example contribution
Datapath starters	ReLU unit, bias adder, multiplier wrapper, accumulator, and simple address counter.
Control starters	Instruction field decoder, state transition cleanup, start/busy/done logic, and reset behavior.
Verification starters	Directed input vectors, expected-result tables, waveform inspection, and boundary-value cases using provided testbench structure.

Organizer notes

- Provide starter interfaces and automated checkers; do not provide completed RTL.
- Require a short explanation or review conversation for each project.
- Pair members when they are stuck, but have each member submit their own implementation.
- Keep the first project modules fixed-width and simple. Introduce advanced SystemVerilog only when the real design needs it.

Sources

HDLBits main page: https://hdlbits.01xz.net/wiki/Main_Page

HDLBits problem sets: https://hdlbits.01xz.net/wiki/Problem_sets

End of guide